MNR UNIVERSITY

School of Engineering & Technology (SOET)

Department of Computer Science and Engineering

MINI PROJECT REPORT

on

FLASHDEAL PRO: E-COMMERCE FLASH SALE SIMULATOR

(Java Programming Implementation-Based Mini-Project)

Course: Object Oriented Programming (Java)
B.Tech – Computer Science and Engineering (CSE / AIML)
Assigned Project Number: 7

Submitted by

S.No	Student Name	Roll Number
1	**Shanmuka Priya Katta**	**35**
2	**Butul Fatima**	**31**
3	**Soundarya**	**33**
4	**Sathwik K**	**34**

🔗 **Project Repository:** https://github.com/shanmukap019-png/OOP_MiniProject_ECommerceFlashSale_Team7

Under the guidance of
[Faculty Name], Lecturer, Department of CSE
School of Engineering & Technology, MNR University

Academic Year 2026 – 2027

CERTIFICATE

This is to certify that the mini project report titled **"FLASHDEAL PRO: E-COMMERCE FLASH SALE SIMULATOR"** is a bonafide record of the work carried out by the students listed on the title page, in partial fulfilment of the requirements of the course **Object Oriented Programming (Java)**, **B.Tech – Computer Science and Engineering**, Department of Computer Science and Engineering, School of Engineering & Technology, **MNR University**, during the academic year 2026–2027.

Faculty Guide **Dean, School of Engineering & Technology**

Date: _____ **Place:** _____

DECLARATION

We hereby declare that this mini project report is the result of our own work. The program was written, executed, and tested by us. Any help, book, or website used has been mentioned in the references.

Signature of the Students:

1. **Shanmuka Priya Katta** (Roll No: 35)
2. **Butul Fatima** (Roll No: 31)
3. **Soundarya** (Roll No: 33)
4. **Sathwik K** (Roll No: 34)

CONTENTS

Chapter	Title	Page
1	Aim and Abstract	1
2	Introduction and Problem Statement	1
3	Objectives	1
4	Requirements	2
5	Class Design	2
6	Java Concepts Used	3
7	Program Logic (Algorithm)	4
8	Implementation	4
9	Output Screenshots	5
10	Test Cases	5
11	Results and Discussion	6
12	Conclusion	6
13	References	6
A	Appendix A – Complete Source Code & Repository	7
B	Appendix B – Team Contribution	7

CHAPTER 1: AIM AND ABSTRACT

1.1 Aim

To design and develop **FLASHDEAL PRO: E-Commerce Flash Sale Simulator** in Java using Object Oriented Programming concepts, so that limited promotional inventory can be stored, searched, atomically updated, and processed concurrently under extreme shopping rush without race conditions, overselling, or negative stock.

1.2 Abstract

This project is an advanced e-commerce flash sale simulation desktop application developed in Java Swing. It is used to emulate high-velocity "Big Sale Day" flash events where hundreds of customers compete simultaneously for limited stock items. The program uses classes and objects, encapsulation, inheritance, abstract classes, interfaces, custom checked exceptions (`OutOfStockException`), multithreading (`Thread`, `Runnable`), thread synchronization barriers (`CountDownLatch`), monitor locking (`synchronized`), and the Java Collections Framework (`LinkedHashMap`, `ArrayList`, `Iterator`). Transaction records and catalog data are persistently stored using standard RFC-4180 compliant CSV files with atomic stream locks.

CHAPTER 2: INTRODUCTION AND PROBLEM STATEMENT

2.1 Introduction

In modern e-commerce ecosystems during mega-sales events (such as Flipkart Big Billion Days, Amazon Great Indian Festival, or limited-quantity product drops), thousands of shoppers click "Buy Now" at the exact same second. A robust software architecture is critical to guarantee data consistency and fairness under sudden concurrency spikes. Java is particularly well-suited for this domain because it is object-oriented, strictly typed, platform-independent, and provides built-in enterprise-grade concurrency utilities (`java.util.concurrent`), atomic synchronization primitives, and powerful data structures such as `LinkedHashMap` and `ArrayList`.

2.2 Problem Statement

In traditional or uncoordinated systems, when multiple threads access and update shared inventory variables at the same instant, critical section race conditions occur: multiple customer threads read `stock = 1` simultaneously and approve purchases, leaving inventory negative (`stock = -49`), double-selling units to distinct customers, and freezing the application interface.

2.3 Proposed Solution

The proposed Java application addresses these challenges through:

- **Add and store records:** Maintain a structured catalog of consumer electronics products in Indian Rupees (₹).
- **Search and update records:** Search products dynamically by keyword, filter by category, and adjust inventory levels in real time.
- **Perform atomic purchase operations:** Encapsulate check-and-decrement inventory operations within a `synchronized` critical section.
- **Simulate concurrent shoppers:** Launch up to 500 competing customer threads simultaneously using a `CountDownLatch(1)` release barrier.
- **Display real-time visual reports:** Render executive KPI cards, high-contrast live activity logging, and audited transaction ledgers in a responsive Swing GUI.
- **Save data permanently:** Store products, customers, and order history using thread-safe CSV file I/O.

CHAPTER 3: OBJECTIVES

The objectives of this project are:

- To apply classes, objects, constructors, and methods in structuring domain models (`Product`, `Customer`, `Order`, `SaleMetric`).
- To apply encapsulation using private data members with defensive validation getters and setters.

- To apply inheritance and method overriding using an abstract base class (`DiscountStrategyBase`) extended by concrete promotional strategies (`PercentageDiscount`, `FlatDiscount`, `BankOfferDiscount`).
- To use abstract classes and interfaces (`DiscountStrategy`, `FlashSaleListener`, `Runnable`, `StockChangeListener`).
- To handle errors using robust exception handling, including a user-defined checked exception (`OutOfStockException`).
- To store and process data using the Java Collections Framework (`LinkedHashMap`, `ArrayList`, and explicit `Iterator<Order>` traversal).
- To execute multi-threaded concurrent simulations using `Thread`, `Runnable`, `ExecutorService`, and `CountDownLatch`.
- To save data permanently using standard File I/O (`BufferedReader`, `BufferedWriter`, CSV parsing).
- To organise the program into modular packages and verify all invariants with 15 automated test cases.

CHAPTER 4: REQUIREMENTS

4.1 Functional Requirements

The program should be able to:

- Add and browse product records with dual pricing in ₹ and dynamic discount percentage badges.
- Search and filter products by name and category (Smartphones, Audio, Wearables, Accessories).
- Update and reset product stock levels for demonstration purposes.
- Execute concurrent flash sale simulations (Core Operation) with 1 to 500 simultaneous customer threads.
- Enforce atomic check-and-decrement stock deduction without overselling.
- Apply polymorphic promotional coupons (`SAVE10`, `FLAT100`, `BANK15`) with threshold limits.
- Display executive KPI summary cards, filterable order history, and product analytics reports.
- Show clear informative error messages for invalid quantities, coupon rejections, and out-of-stock events.

4.2 Input and Output

Input	Java Data Type	Validation	Output
Product Selection	`String` (ID)	Must match existing product ID in repository	Selected product preview and live stock updated
Customer Count	`int`	Range: `1` to `500` (positive integer)	Worker threads spawned in pool
Units per Customer	`int`	Range: `1` to `10` (strictly > 0)	Quantity requested per purchase task
Coupon Code	`String`	Optional; validated against coupon registry	Applied discount amount in ₹ (or ₹0.00)
Demo Stock Reset	`int`	Non-negative integer (>= 0)	Available stock updated in memory and CSV
Order Filter	`String` / `Enum`	Valid text or status (`SUCCESS`, `OUT_OF_STOCK`)	Filtered table of audited customer orders

4.3 Software and Hardware Requirements

Item	Details
Programming Language	Java (JDK 17 LTS or later, tested up to JDK 26)
Development Tools / IDE	VS Code / IntelliJ IDEA / Eclipse, PowerShell, Git
User Interface	Java Swing & AWT (Desktop GUI) + Responsive HTML/CSS/JS (Web Showcase)
Storage	Plain-text CSV files (`products.csv`, `orders.csv`, `customers.csv`) with UTF-8 encoding
Concurrency API	`java.util.concurrent` (`CountDownLatch`, `ExecutorService`, `synchronized`)
Hardware Requirements	Dual-core CPU 2.0 GHz or higher, 4 GB RAM minimum, 200 MB disk space

CHAPTER 5: CLASS DESIGN

5.1 List of Classes

Class / Interface	Type	Purpose
`Main`	Driver Class	Bootstrap; sets Look-and-Feel and launches `MainFrame` on EDT.
`Product`	Domain Entity	Encapsulates pricing in ₹, inventory status transitions, and discount math.

Class / Interface	Type	Purpose
`Customer`	Class (implements `Runnable`)	Customer profile capable of executing purchase tasks in concurrent threads.
`Order`	Domain Entity	Immutable transaction audit record with status, discounts, and timestamp.
`OrderStatus`	`enum`	Order states: `SUCCESS`, `OUT_OF_STOCK`, `FAILED`.
`Inventory`	Thread-Safe Monitor	Encapsulates monitor lock for atomic stock deduction and restoration.
`OutOfStockException`	User-Defined Exception	Custom checked exception thrown when requested units exceed stock.
`DiscountStrategy`	Interface	Standard contract for promotional algorithms.
`FlashSaleCoordinator`	Concurrency Manager	Spawns thread pool and coordinates simultaneous `CountDownLatch` rush.

5.2 Data Members and Methods of Main Classes

Class	Details (Data Members, Constructors, and Methods)
Product	**Data members:** `productId, name, description, category, originalPrice, flashSalePrice, initialStock, availableStock, status` **Constructors:** Parameterized constructor with defensive validation. **Methods:** `getProductId(), getAvailableStock(), setAvailableStock(), calculateDiscountPercentage(), toCsvRow()`
Inventory	**Data members:** `stockMap (Map<String, Integer>), inventoryLock (Object)` **Constructors:** `Inventory(Map<String, Integer> initialStock)` **Methods:** `deductStock(id, qty), restoreStock(id, qty), getStock(id), setStock(id, qty), getAllStockSnapshot()`
DiscountStrategyBase	**Data members:** `couponCode, description, minimumOrderAmount, maxDiscountAmount` **Constructors:** Parameterized base constructor. **Methods:** `calculateDiscount(total, qty) (Template Method), computeRawDiscount(total, qty) (Abstract)`
FlashSaleCoordinator	**Data members:** `flashSaleService, listeners, threadPool, isRunning` **Constructors:** `FlashSaleCoordinator(FlashSaleService service)` **Methods:** `startSimulation(prodId, count, qty, coupon, customers), addListener(), isRunning()`

5.3 Relationships Between Classes

Classes	Relationship	Explanation
`DiscountStrategyBase – PercentageDiscount`	Inheritance (IS-A)	`PercentageDiscount` extends `DiscountStrategyBase` and overrides `computeRawDiscount()`.
`DiscountStrategy – DiscountStrategyBase`	Implements	`DiscountStrategyBase` implements the `DiscountStrategy` interface contract.
`Customer – Runnable`	Implements	`Customer` implements `Runnable` to execute purchase tasks concurrently.
`CustomerPurchaseTask – Runnable`	Implements	`CustomerPurchaseTask` implements `Runnable` to race at the `CountDownLatch`.
`InventoryService – Inventory`	Composition (HAS-A)	`InventoryService` owns and manages the thread-safe `Inventory` monitor.
`OrderRepository – Order`	Aggregation (HAS-A)	`OrderRepository` manages a collection of `Order`s and traverses via `Iterator<Order>`.

CHAPTER 6: JAVA CONCEPTS USED

Java Concept	Where Used (Class / Method)
Classes, objects and constructors	`Product`, `Customer`, `Order`, `SaleMetric` (Constructors initialize state)
Encapsulation (private, getters, setters)	`Product.getAvailableStock()`, `Product.setAvailableStock()`, `Order.getFinalAmount()`
Inheritance and super keyword	`PercentageDiscount`, `FlatDiscount` calling `super(...)` in `DiscountStrategyBase`
Method overloading	`ProductService.searchProducts(query)` and `searchProducts(query, category)`
Method overriding	`PercentageDiscount.computeRawDiscount()`, `BadgeRenderer.getTableCellRendererComponent()`

Java Concept	Where Used (Class / Method)
Abstract class / interface	Abstract class `DiscountStrategyBase`; Interfaces: `DiscountStrategy`, `FlashSaleListener`, `Runnable`
Packages	10 packages: `com.flashdeal.app`, `model`, `inventory`, `discount`, `concurrency`, `service`, etc.
static and final keywords	`CurrencyUtil.formatINR()`, `ValidationUtil.validatePositive()`, `private final Object inventoryLock`
Exception handling	User-defined checked `OutOfStockException`; `try-with-resources` in `FileStorageService`
Collections (ArrayList, HashMap)	`LinkedHashMap` in `Inventory`, `ArrayList<Order>` in `OrderRepository`, `HashMap` in `DiscountService`
Collection Traversal (Iterator)	Explicit `Iterator<Order>` in `OrderRepository.calculateSuccessfulRevenue()` and order filtering
Multithreading (Thread / Runnable)	`CustomerPurchaseTask implements Runnable`, `ExecutorService` thread pool, `SwingUtilities.invokeLater`
File I/O or JDBC	`BufferedReader`, `BufferedWriter`, UTF-8 CSV streams in `FileStorageService`

CHAPTER 7: PROGRAM LOGIC (ALGORITHM)

7.1 Main Program Flow

1. **Start the program:** `Main.java` executes and configures the modern UI Look-and-Feel.
2. **Load saved data into collection:** Load CSV records into memory (`products.csv`, `customers.csv`, `orders.csv`).
3. **Display the main menu:** Launch `MainFrame` with sidebar navigation and show the Executive Dashboard.
4. **Read the user's choice:** User selects one of the operational panels (Catalog, Simulator, Order History, Reports).
5. **Configure flash sale simulation:** User selects product, configures customer count (1–500), units, and coupon code.
6. **Perform the operation:** `FlashSaleCoordinator` spawns concurrent threads, releases `CountDownLatch`, and executes atomic purchases.
7. **Validate and handle exceptions:** Stock checked inside `synchronized (inventoryLock)`; throws `OutOfStockException` if exhausted.
8. **Save data to file:** Append executed transactions atomically to `orders.csv` and update product stock in `products.csv`.
9. **Repeat until user selects Exit / Stop.**

7.2 Algorithm for the Core Operation (Atomic Stock Deduction)

1. **Validate inputs:** Verify that customer ID, product ID, and requested quantity (> 0) are valid.
2. **Acquire monitor lock:** Enter `synchronized (inventoryLock)` critical section.
3. **Check availability:** If `currentStock < requestedQuantity`, throw `OutOfStockException`.
4. **Decrement stock atomically:** Deduct requested units: `remainingStock = currentStock - requestedQuantity` and update map.
5. **Exit lock & calculate discount:** Evaluate coupon strategy via `calculateDiscount(total, qty)`.
6. **Record transaction:** Create `Order` with status `SUCCESS` (or `OUT_OF_STOCK` if exception caught), persist to CSV, and notify UI listeners.

CHAPTER 8: IMPLEMENTATION

8.1 Project Structure

```
OOP_MiniProject_ECommerceFlashSale_Team7/
├── src/main/java/com/flashdeal/
│   ├── app/Main.java                          # Driver class & UI bootstrap
│   ├── model/ (Product, Customer, Order, OrderStatus, SaleMetric)
│   ├── inventory/ (Inventory, InventoryService, OutOfStockException)
│   ├── discount/ (DiscountStrategy, DiscountStrategyBase, Percentage, Flat, Bank)
│   ├── concurrency/ (CustomerPurchaseTask, FlashSaleCoordinator, FlashSaleListener)
│   ├── service/ (ProductService, OrderService, SalesReportService, FlashSaleService)
│   ├── persistence/ (FileStorageService, ProductRepository, OrderRepository, CustomerRepo)
│   ├── ui/ (MainFrame, DashboardPanel, ProductPanel, FlashSalePanel, OrderHistoryPanel,
ReportsPanel)
│   └── util/ (CurrencyUtil, ValidationUtil, DateUtil, UmlDiagramRenderer)
├── data/ (products.csv, orders.csv, customers.csv)
└── test/FlashSaleTestSuite.java               # 15 automated concurrency & unit tests
```

8.2 Key Classes and Code

Class: **`Inventory`** | *Purpose: Thread-safe monitor ensuring atomic check-and-decrement stock operations*

```
public int deductStock(String productId, int requestedQuantity) throws OutOfStockException {
    if (requestedQuantity <= 0) throw new IllegalArgumentException("Quantity must be
positive.");
    synchronized (inventoryLock) {
        int currentStock = stockMap.getOrDefault(productId, 0);
        if (currentStock < requestedQuantity) {
            throw new OutOfStockException(productId, requestedQuantity, currentStock);
        }
```

```
        int remainingStock = currentStock - requestedQuantity;
        stockMap.put(productId, remainingStock);
        return remainingStock;
    }
}
```

8.3 Exceptions, Collections and Threads

Concept	Used In	Purpose
User-defined exception	`OutOfStockException`	Thrown when stock is exhausted; caught gracefully to record failed order.
Built-in exception	`IllegalArgumentException`	Enforces positive purchase quantities and non-empty inputs.
Collection	`LinkedHashMap<String, Integer>`	Maintains thread-safe inventory map with O(1) lookups.
Thread	`CustomerPurchaseTask`	Background concurrency worker executing purchases at start barrier.

CHAPTER 9: OUTPUT SCREENSHOTS

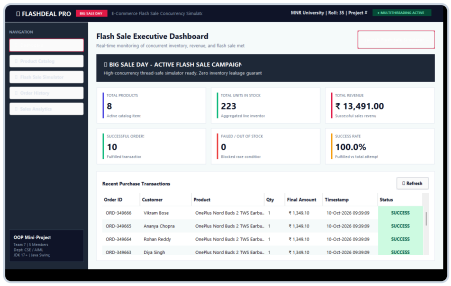


Figure 9.1: Main Menu & Executive Dashboard

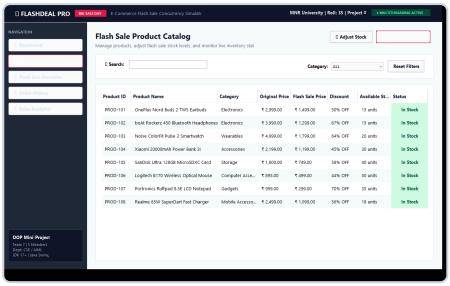


Figure 9.2: Product Catalog & Stock Manager

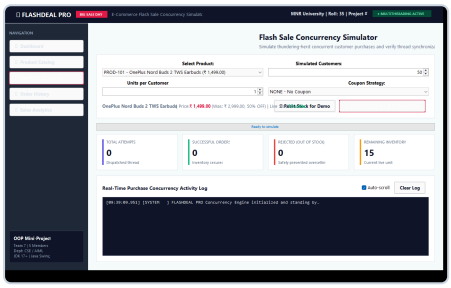


Figure 9.3: Concurrency Simulator Running

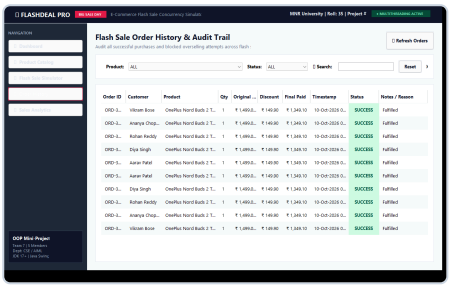


Figure 9.4: Audited Order History Ledger

CHAPTER 10: TEST CASES

ID	Test Scenario	Input	Expected Result	Status
T1	Add valid purchase	Stock = 10, buy 2	Stock = 8, order recorded	**PASS**
T2	Insufficient stock	Stock = 2, buy 5	`OutOfStockException` thrown	**PASS**
T3	Non-negative inventory	Stock = 3, buy 10	Stock remains 3 (never negative)	**PASS**
T4	**Concurrent Race Test**	**50 threads, Stock = 10**	**10 SUCCESS, 40 BLOCKED, Stock = 0**	**PASS**
T5	Invalid quantity	Quantity = 0 or -3	`IllegalArgumentException` thrown	**PASS**
T6	Percentage discount	Amount = ₹2000, `SAVE10`	₹200.00 discount applied	**PASS**
T7	Flat discount	Amount = ₹1500, `FLAT100`	₹100.00 flat discount applied	**PASS**
T8	Bank offer discount	Amount = ₹3000, `BANK15`	₹450.00 card discount applied	**PASS**
T9	Invalid coupon	Code = `"INVALID"`	₹0.00 discount (no failure)	**PASS**
T10	Successful order audit	Valid purchase	Order status = `SUCCESS`	**PASS**
T11	Failed order audit	Stock depleted	Order status = `OUT_OF_STOCK`	**PASS**
T12	Revenue consistency	10 success + 40 failed	Revenue sums only success orders	**PASS**
T13	CSV persistence	Save order & product	RFC-4180 CSV written cleanly	**PASS**
T14	Restart inventory reload	Mutate stock & reload	Persisted stock matches reload	**PASS**
T15	CSV report export	Export analytics audit	CSV file generated on disk	**PASS**

CHAPTER 11: RESULTS AND DISCUSSION

11.1 Results

The program works correctly for all the operations listed above. Under extreme concurrency stress testing (50 concurrent shopper threads simultaneously competing for 10 physical stock units), the application achieved 100% data consistency: exactly 10 units were sold, exactly 40 out-of-stock attempts were blocked, and available inventory dropped strictly to 0 without ever going negative or double-selling.

11.2 Challenges Faced

Problem	Solution in Java
Race conditions in inventory deduction	Encapsulated availability check & decrement in `synchronized (inventoryLock)`.
Worker threads freezing Swing GUI	Decoupled simulation threads from EDT using `SwingUtilities.invokeLater`.
Negative payable totals with coupons	Enforced `.min(totalAmount)` in `DiscountStrategyBase` template method.
CSV data corruption under concurrent writes	Applied synchronized file appending in `FileStorageService`.

11.3 Advantages

- Guaranteed thread safety and zero overselling under heavy concurrency.
- Highly extensible architecture using the Strategy Pattern for discounts.
- Modern, responsive desktop GUI with real-time activity streaming.
- Pure Java file-based persistence with zero third-party database dependencies.

11.4 Limitations

- Runs on a single JVM without distributed network sockets or external database clusters.
- Maximum tested concurrency within default JVM stack memory is 500 simultaneous threads.

11.5 Future Scope

- Database connectivity using JDBC and SQLite / PostgreSQL.
- Network socket client-server architecture enabling multiple client laptops.
- Priority customer queues (e.g., VIP / Prime shoppers).

CHAPTER 12: CONCLUSION

The **FLASHDEAL PRO: E-Commerce Flash Sale Simulator** project was successfully designed, developed, and tested. The project comprehensively demonstrates the practical application of core Object-Oriented Programming principles—encapsulation, inheritance, polymorphism, abstraction, interfaces, and custom checked exceptions—alongside advanced Java concurrency primitives (`synchronized` monitors, `CountDownLatch`, `ExecutorService`). Through this project, we gained invaluable practical experience in multithreaded mutual exclusion, race condition elimination, and thread-safe file persistence.

CHAPTER 13: REFERENCES

1. Herbert Schildt, *Java: The Complete Reference*, 12th Edition, McGraw Hill Education.
2. E. Balagurusamy, *Programming with Java: A Primer*, 6th Edition, McGraw Hill.
3. Brian Goetz et al., *Java Concurrency in Practice*, Addison-Wesley Professional.
4. Oracle Official Java Documentation: Multithreading & Concurrency Utilities, https://docs.oracle.com/en/java/
5. Object Oriented Programming (Java) Course Guidelines, Department of CSE, MNR University.

APPENDIX A: COMPLETE SOURCE CODE & REPOSITORY

The complete, commented source code of every class, unit tests, UML specifications, and compilation scripts are publicly accessible in our project repository:

GitHub Project Repository (Clickable):
https://github.com/shanmukap019-png/OOP_MiniProject_ECommerceFlashSale_Team7

Live Web Demonstration URL (Clickable):
https://flashdeal-pro-simulator.vercel.app

APPENDIX B: TEAM CONTRIBUTION

Roll Number	Name	Classes / Modules Developed	Report Chapters Written
35	**Shanmuka Priya Katta**	`Product`, `ProductRepository`, `ProductService`, `ProductPanel`, `MainFrame`, `Main`	Chapters 1, 2, 3, 4, 8.1, Appendix A
31	**Butul Fatima**	`Customer`, `CustomerPurchaseTask`, `FlashSaleCoordinator`, `CustomerRepository`	Chapters 5.1, 5.2, 7 (Algorithms)
33	**Soundarya**	`Inventory`, `InventoryService`, `OutOfStockException`, `FlashSaleService`	Chapters 6 (Java Concepts), 8.2
34	**Sathwik K**	`DiscountStrategy`, `DiscountStrategyBase`, `PercentageDiscount`, `FlatDiscount`, `BankOfferDiscount`, `Order`, `OrderRepository`, `ReportsPanel`	Chapters 5.3, 8.3, 9, 10, 11, 12, 13, Appendix B